回课: 控制流

机器级编程 II

Rainybunny

2026 年 4 月 1 日

目录

1 C loops 与 goto loops 的互译

2 switch 与跳转表

3 补充: 跳转的相关细节

1 C loops 与 goto loops 的互译

2 switch 与跳转表

3 补充: 跳转的相关细节

do-while loop

```
do {
    count += x & 1;
    x >>= 1;
} while (x);
```

$\Updownarrow$

```
loop:
    count += x & 1;
    x >>= 1;
    if (x) goto loop;
```

while loop

```
1    while (x) {
2      count += x & 1;
3      x >>= 1;
4    }
```

$$\Updownarrow$$

方案一: jump to middle.

```
1    goto check;
2  loop:
3    count += x & 1;
4    x >>= 1;
5  check:
6    if (x) goto loop;
```

while loop

```
1    while (x) {
2      count += x & 1;
3      x >>= 1;
4    }
```

$$\Updownarrow$$

方案二: guarded-do.

```
1    if (!x) goto finished;
2  loop:
3    count += x & 1;
4    x >>= 1;
5    if (x) goto loop;
6  finished: ;
```

while loop

在不同优化级别下, `gcc` 对 `while` 循环的处理方式不完全相同. 在实验[1]中:

- 启用 `-Og` 选项, `while` 被处理为 jump-to-middle 方案;
- 启用 `-O1` 选项, `while` 被处理为 guarded-do 方案.

相比 guarded-do, jump-to-middle 会多一次跳转 (第一次成功进入循环体之前会连续跳转两次), 有轻微性能劣势.

[1] https://godbolt.org/z/7KKMWs3Mz

for loop

```
1    for (int i = 0; x >> i; ++i) {
2      count += x >> i & 1;
3    }
```

$$\Updownarrow$$

```
1    int i = 0;
2    if (!(x >> i)) goto finished;
3  loop:
4    count += x >> i & 1;
5    ++i;
6    if (x >> i) goto loop;
7  finished: ;
```

多重循环

```
  movl   $1000000000, 96(%rsp)
  movl   $9, %esi
  movl   $0, %eax
  jmp .L4
.L6:
  subl   $1, %eax
.L5:
  movslq  %eax, %rdx
  cmpl   %ecx, 96(%rsp,%rdx,4)
  jle .L6
  addl   $1, %eax
  movslq  %eax, %rdx
  movl   %ecx, 96(%rsp,%rdx,4)
  movslq  %esi, %rdx
  movl   %eax, 48(%rsp,%rdx,4)
  subl   $1, %esi
.L4:
  testl  %esi, %esi
  js .L12
  movslq  %esi, %rdx
  movl   (%rsp,%rdx,4), %ecx
  jmp .L5
.L12:
  ; (omitted)
```

多重循环

```
  movl   $1000000000, 96(%rsp) ; stk[0] = 1e9
  movl   $9, %esi              ; i = 9
  movl   $0, %eax              ; top = 0
  jmp .L4
.L6:
  subl   $1, %eax              ; --top
.L5:
  movslq  %eax, %rdx
  cmpl   %ecx, 96(%rsp,%rdx,4) ; x : stk[top]
  jle .L6                      ; jump if stk[top] <= x
  addl   $1, %eax              ; ++top
  movslq  %eax, %rdx
  movl   %ecx, 96(%rsp,%rdx,4) ; stk[top] = x
  movslq  %esi, %rdx
  movl   %eax, 48(%rsp,%rdx,4) ; lgis[i] = top (length of greedy LIS)
  subl   $1, %esi              ; --i
.L4:
  testl  %esi, %esi
  js .L12                      ; NOT jump if i >= 0
  movslq  %esi, %rdx
  movl   (%rsp,%rdx,4), %ecx   ; x = ary[i]
  jmp .L5
.L12:
  ; (omitted)
```

1 C loops 与 goto loops 的互译

2 switch 与跳转表

3 补充: 跳转的相关细节

C 中的 jump table

```c
switch (x) {
  case 0:
    x = 114;
  case 1:
    x += 514;
    break;
  case 3:
  case 4:
    x = 1919;
    break;
  default:
    x = 810;
}
```

⇕

```c
void bar(int& x) {
  void* jmpt[] = { &&loc_0, &&loc_1, &&loc_def, &&loc_3_4, &&loc_3_4 };
  if ((unsigned)x > 4) goto loc_def;
  goto *jmpt[x];
loc_0:
  x = 114;
loc_1:
  x += 514;
  goto done;
loc_3_4:
  x = 1919;
loc_def:
  x = 810;
done:;
}
```

汇编中的 jump table

```
iterate_8per(int (*)(int), int, int):
  movl    %esi, %eax
  pushq   %r12
  movl    %esi, %r12d
  andl    $7, %eax
  pushq   %rbp
  pushq   %rbx
  movq    %rdi, %rbx
  movl    %edx, %edi
  jmp     *.L21(,%rax,8)
.L21:
  .quad   .L19
  .quad   .L27
  .quad   .L26
  .quad   .L25
  .quad   .L24
  .quad   .L23
  .quad   .L22
  .quad   .L20
.L20:
  call    *%rbx
  movl    %eax, %edi
.L22: ; two same lines
.L23: ; two same lines
.L24: ; two same lines
.L25: ; two same lines
.L26: ; two same lines
.L27: ; two same lines
  ; (omitted)
```

汇编中的 jump table

```
iterate_8per(int (*)(int), int, int):
  movl    %esi, %eax
  pushq   %r12
  movl    %esi, %r12d
  andl    $7, %eax
  pushq   %rbp
  pushq   %rbx
  movq    %rdi, %rbx
  movl    %edx, %edi
  jmp     *.L21(,%rax,8)   ; switch here
.L21:                       ; jump table for switch
  .quad .L19
  .quad .L27
  .quad .L26
  .quad .L25
  .quad .L24
  .quad .L23
  .quad .L22
  .quad .L20            ; targets for different cases
.L20:
  call    *%rbx
  movl    %eax, %edi
.L22: ; two same lines
.L23: ; two same lines
.L24: ; two same lines
.L25: ; two same lines
.L26: ; two same lines
.L27: ; two same lines
  ; (omitted)
```

引入 jump table 的优劣

- 优点:
  - 执行跳转指令的次数从 $\mathcal{O}(n)$ 降为 $\mathcal{O}(1)$, 适合 case 较多且分布较为密集的 switch.
- 缺点:
  - 需要额外的内存空间存储 jump table.
  - case 分布过于稀疏时, jump table 会浪费大量空间.
  - case 较少时, 直接使用条件测试 + 固定标签跳转反而更快.

[2] https://godbolt.org/z/8Y3zznaME

引入 jump table 的优劣

- 优点:
  - 执行跳转指令的次数从 $\mathcal{O}(n)$ 降为 $\mathcal{O}(1)$, 适合 case 较多且分布较为密集的 switch.
- 缺点:
  - 需要额外的内存空间存储 jump table.
  - case 分布过于稀疏时, jump table 会浪费大量空间.
  - case 较少时, 直接使用条件测试 + 固定标签跳转反而更快.

gcc 会根据 case 数量和分布情况决定是否使用 jump table.[2]

[2] https://godbolt.org/z/8Y3zznaME

switch 语句的 if-else 翻译

另一种 switch 的代码实现方式: if-else 链. 我们可以将所有 case 的内部语句依据 `break` 划分为若干个代码块, 显然程序最多进入一个代码块, 这可以用最外层的 if-else 链来控制.

switch 语句的 if-else 翻译

另一种 switch 的代码实现方式: if-else 链. 我们可以将所有 case 的内部语句依据 `break` 划分为若干个代码块, 显然程序最多进入一个代码块, 这可以用最外层的 if-else 链来控制.

在一个代码块内, 由于没有 `break`, 程序会从某处开始顺序执行后续代码块的内容. 为此, 我们维护一个布尔变量, 记录 switch 变量是否已经触发过进入这个代码块的条件, 以决定代码块中的每条语句是否执行.

switch 语句的 if-else 翻译

```
switch (x) {
  case 0: x += 5;
  case 1: x += 2;
  case 2: x *= 9; break;
  case 3: x *= 7;
  case 4: x *= 4;
  default: x -= 8;
}
```

⇕

```
int sx = x; // switch 内部语句可能修改 x
bool entered = false;
if (sx == 0 || sx == 1 || sx == 2) {
  if (entered |= sx == 0) x += 5;
  if (entered |= sx == 1) x += 2;
  x *= 9; // 已知进入此代码块，所以一定会执行
} else { // default 所在块：无须再判断条件...
  if (entered |= sx == 3) x *= 7;
  if (entered |= sx == 4) x *= 4;
  x -= 8; // ...因为这句话一定需要执行
}
```

1 C loops 与 goto loops 的互译

2 switch 与跳转表

3 补充：跳转的相关细节

跳转指令的 PC 相对编码

```
0000000000000000 <popcount>:
   0: f3 0f 1e fa            endbr64
   4: 55                     push   %rbp
   5: 48 89 e5               mov    %rsp,%rbp
   8: 89 7d ec               mov    %edi,-0x14(%rbp)
   b: c7 45 fc 00 00 00 00   movl   $0x0,-0x4(%rbp)
  12: eb 13                  jmp    27 <popcount+0x27>
  14: 8b 45 ec               mov    -0x14(%rbp),%eax
  17: 83 e0 01               and    $0x1,%eax
  1a: 89 c2                  mov    %eax,%edx
  1c: 8b 45 fc               mov    -0x4(%rbp),%eax
  1f: 01 d0                  add    %edx,%eax
  21: 89 45 fc               mov    %eax,-0x4(%rbp)
  24: d1 6d ec               shrl   -0x14(%rbp)
  27: 83 7d ec 00            cmpl   $0x0,-0x14(%rbp)
  2b: 75 e7                  jne    14 <popcount+0x14>
  2d: 8b 45 fc               mov    -0x4(%rbp),%eax
  30: 5d                     pop    %rbp
  31: c3                     ret
```

跳转指令的 PC 相对编码

```
0000000000000000 <popcount>:
   0: f3 0f 1e fa            endbr64
   4: 55                     push   %rbp
   5: 48 89 e5               mov    %rsp,%rbp
   8: 89 7d ec               mov    %edi,-0x14(%rbp)
   b: c7 45 fc 00 00 00 00   movl   $0x0,-0x4(%rbp)
  12: eb 13                  jmp    27 <popcount+0x27>
  14: 8b 45 ec               mov    -0x14(%rbp),%eax
  17: 83 e0 01               and    $0x1,%eax
  1a: 89 c2                  mov    %eax,%edx
  1c: 8b 45 fc               mov    -0x4(%rbp),%eax
  1f: 01 d0                  add    %edx,%eax
  21: 89 45 fc               mov    %eax,-0x4(%rbp)
  24: d1 6d ec               shrl   -0x14(%rbp)
  27: 83 7d ec 00            cmpl   $0x0,-0x14(%rbp)
  2b: 75 e7                  jne    14 <popcount+0x14>
  2d: 8b 45 fc               mov    -0x4(%rbp),%eax
  30: 5d                     pop    %rbp
  31: c3                     ret
```

$\Longrightarrow$ 一般的跳转指令使用相对编码，计算 jmp 的**下一条指令地址**与目标指令地址的差值，将差值编码为一个 1/2/4 byte(s) 的**补码**.

跳转指令的 PC 相对编码

```
0000000000000000 <popcount>:
   0: f3 0f 1e fa            endbr64
   4: 55                     push   %rbp
   5: 48 89 e5               mov    %rsp,%rbp
   8: 89 7d ec               mov    %edi,-0x14(%rbp)
   b: c7 45 fc 00 00 00 00   movl   $0x0,-0x4(%rbp)
  12: eb 13                  jmp    27 <popcount+0x27>
  14: 8b 45 ec               mov    -0x14(%rbp),%eax
  17: 83 e0 01               and    $0x1,%eax
  1a: 89 c2                  mov    %eax,%edx
  1c: 8b 45 fc               mov    -0x4(%rbp),%eax
  1f: 01 d0                  add    %edx,%eax
  21: 89 45 fc               mov    %eax,-0x4(%rbp)
  24: d1 6d ec               shrl   -0x14(%rbp)
  27: 83 7d ec 00            cmpl   $0x0,-0x14(%rbp)
  2b: 75 e7                  jne    14 <popcount+0x14>
  2d: 8b 45 fc               mov    -0x4(%rbp),%eax
  30: 5d                     pop    %rbp
  31: c3                     ret
```

$\Longrightarrow$ 一般的跳转指令使用相对编码，计算 jmp 的**下一条指令地址**与目标指令地址的差值，将差值编码为一个 1/2/4 byte(s) 的**补码**.

c.f. 绝对地址跳转：`jmp *%rax`, `jmp *(%rax)`, ...

使用条件传递消除跳转

跳转指令会破坏指令流水线，影响 CPU 的指令级并行执行效率. 因此，在某些情况下，我们可以使用条件传递 (conditional move) 指令来消除跳转.

[3] 见 CS:APP p.379; 这里的目标和结论都偏经验.

使用条件传递消除跳转

跳转指令会破坏指令流水线，影响 CPU 的指令级并行执行效率. 因此，在某些情况下，我们可以使用条件传递 (conditional move) 指令来消除跳转.

这鼓励我们写出”条件传递风格”的代码[3].

[3] 见 CS:APP p.379; 这里的目标和结论都偏经验.

使用条件传递消除跳转

跳转指令会破坏指令流水线，影响 CPU 的指令级并行执行效率. 因此，在某些情况下，我们可以使用条件传递 (conditional move) 指令来消除跳转.

这鼓励我们写出”条件传递风格”的代码[3].

2× faster 冒泡排序：https://godbolt.org/z/YTG3sEqTd (彩蛋：在这个例子中，你还将看到 a. 笨笨编译器在函数末尾放两个 `ret` 用两个 label 跳转；b. `-O3` 默认的 slp-vectorize 昏招频出带来一倍负优化.)

[3] 见 CS:APP p.379；这里的目标和结论都偏经验.

&&, || 与 goto

`&&` 和 `||` 总是短路求值的，这意味着在某个表达式求值时，我们总能假定前面的子表达式已经求值完毕且均为真/假.

[4] https://godbolt.org/z/zMz4s3aTP

&&, || 与 goto

`&&` 和 `||` 总是短路求值的，这意味着在某个表达式求值时，我们总能假定前面的子表达式已经求值完毕且均为真/假.

这类似于若干个 if 语句的嵌套，编译器一般也会将其翻译为这样的若干跳转指令.

[4] https://godbolt.org/z/zMz4s3aTP

&&, || 与 goto

`&&` 和 `||` 总是短路求值的，这意味着在某个表达式求值时，我们总能假定前面的子表达式已经求值完毕且均为真/假.

这类似于若干个 if 语句的嵌套，编译器一般也会将其翻译为这样的若干跳转指令.

不过，类似于编译器在已知求值简单时引入 `cmov` 这一优化，编译器 (语义上) 也可能会将 `&&` 和 `||` 翻译不短路的 `&` 和 `|`，以减少跳转指令的使用.[4]

[4] https://godbolt.org/z/zMz4s3aTP

Thanks!